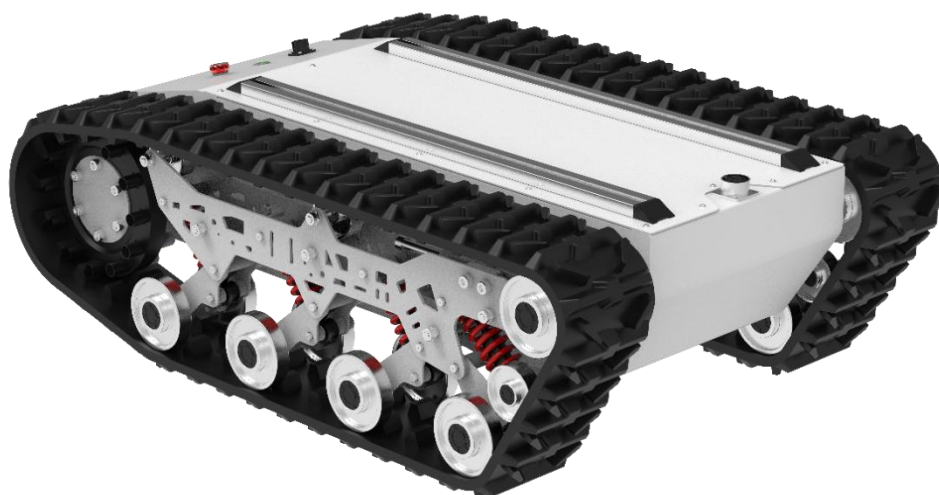


YUHESEN

TK-mid 中型履带线控底盘

ROS 驱动包使用说明书

User manual V1.1.1



深圳煜禾森科技有限公司
Shenzhen Yuhesen Technology Co., Ltd.

目 录

1. 前言 Foreword	2
重要安全信息 Safety Information	3
2. CAN 卡配置	5
2.1. 检查 CAN 是否正常	5
2.2. 手动启动 CAN 卡设置	6
2.3. 开机自启设置	6
2.4. 通讯测试	7
3. ROS 驱动包使用	9
3.1. 编译	9
3.2. 运行	9
3.3. 测试	10
4. 话题说明	11
4.1.发布话题	11
4.2.订阅话题	12
5. 注意事项 Attention	18

1. 前言 Foreword

- (1) 非常感谢您购买我司产品，本用户手册适用于 TK-mid 智能机器人移动底盘（以下简称“TK-mid”）。
- (2) 使用之前请务必先认真阅读本使用手册及注意事项，并严格遵守本说明，正确使用。
- (3) 在严重违反本使用说明的情况下使用本产品造成的损失我司将不承担相关责任。
- (4) 请妥善保管本说明书以便于您操作时随时翻阅参照使用。
- (5) 应请专业人员对底盘设备进行调试、连接、安装，避免造成不可挽回的损失。
- (6) 请勿在带电情况进行安装、移除或更换设备线路。如确实需要带电调试本产品，请选用绝缘良好的专用调试工具。
- (7) 请在法律法规允许的条件下使用本产品，不会影响到公共财产和生命安全。
- (8) 我司对该产品将会不定期更新迭代，更新的内容将会在新版手册中加入，恕不另行通知。
- (9) 本手册可能包含技术上不准确的地方或与产品操作不相符的地方，如果您按照本手册使用时遇到无法解决的问题，请与我司客服或技术支持部门取得联系。
- (10) 关于本手册内容我们力保正确无误，如您发现有不妥或错误，请与我们联系确认，谢谢！

重要安全信息 Safety Information

本手册中的信息不包含设计、安装和操作一个完整的机器人应用，也不包含所有可能对这一完整的系统的安全造成影响的周边设备。该完整的系统的设计和使用需符合该机器人安装所在国的标准和规范中确立的安全要求。TK-mid 的集成商和终端客户有责任确保遵循相关国家的切实可行的法律法规，确保完整的机器人应用实例中不存在任何重大危险。这包括但不限于以下内容：

■ .有效性和责任：

- 对完整的机器人系统做一个风险评估。将风险评估定义的其他机械的附加安全设备连接在一起。确认整个机器人系统的外围设备包括软件和硬件系统的设计和安装准确无误。
- 本机器人不具备一个完整的自主移动机器人具备的自动防撞、防跌落、生物接近预警等相关安全功能但不局限于上述描述，相关功能需要集成商和终端客户遵循相关规定和切实可行的法律法规进行安全评估，确保开发完成的机器人在实际应用中不存在任何重大危险和安全隐患。
- 收集技术文件中的所有文档：包括风险评估和本手册。在操作和使用设备之前已经知晓可能存在的安全风险。

■ .环境：

- 首次使用，请先仔细阅读本手册，了解基本操作内容与操作规范。
- 遥控操作，选择相对空旷区域使用，车上本身是不带任何自动避障传感器。
- 在-20℃~50℃的环境温度中使用。
- 车辆非单独定制 IP 防护等级，车辆防水、防尘能力为 IP64。

■ .检查：

- 检查确保各设备的电量充足。确保车辆无明显异常。检查遥控器的电池电量是否充足。

■ .操作：

- 保证操作时周围区域相对空旷。在视距内遥控控制。

- TK-mid 最大的载重为 80KG，在使用时，确保有效载荷不超过 80KG。
- 当设备低电量报警时请及时充电。当设备出现异常时，请立即停止使用，避免造成二次伤害。
- 当设备出现异常时，请联系相关技术人员，请勿擅自处理。
- 请根据设备的 IP 防护等级在满足防护等级要求的环境中使用。
- 请勿直接推车。
- 充电时，确保周围环境温度大于 0℃。

■ .保养：

- 轮胎磨损严重，请及时更换。
- 如果长时间不使用电池，满电情况下需要按照每个月对电池进行周期性充电。
- 电池一个月之内至少进行一次充电。

2. CAN 卡配置

ROS 驱动包使用前提条件：

- (1) ubuntu22.04 系统
- (2) ubuntu 装好 ros2 humble
- (3) 电脑平台：x86_64、amd

注意：如果为 arm 架构没有识别到 can 卡，请联系售后，安装 pcan 驱动。

建议不要使用虚拟机，直接安装 ubuntu 系统使用

2.1.检查 CAN 是否正常

- (1) 将 CAN 卡接到电脑的 USB 口



- (2) 打开终端，输入命令 `lsusb` 在输出信息中看到以下内容，则表示 CAN 能够正常使用

```
wh@wh-KLVF-XX:~$ lsusb
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 003: ID 2b7e:b567 Kingcome HD Camera
Bus 003 Device 002: ID 275d:0ba6 USB OPTICAL MOUSE
Bus 003 Device 004: ID 8087:0026 Intel Corp.
Bus 003 Device 008: ID 0c72:000c PEAK System PCAN-USB
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

2.2. 手动启动 CAN 卡设置

(1) 重新拔插 CAN 卡后，需要执行以下命令使能 CAN 卡：

```
sudo ip link set can0 type can bitrate 500000
```

```
sudo ip link set can0 up
```

注意：检查自己电脑给分配的是否为 can0 接口（可以用 ifconfig 命令查看，如果提示没有这个指令，按照提示安装即可），自带 can 的电脑分配的 can 端口不一定是 can0。

```
yhs@yhs-ros:~$ ifconfig -a
can0: flags=193<UP,RUNNING,NOARP> mtu 16
      unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (
UNSPEC)
      RX packets 780360 bytes 6242880 (6.2 MB)
      RX errors 0 dropped 52009 overruns 0 frame 0
      TX packets 0 bytes 0 (0.0 B)
      TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

(2) 看到 can 卡绿灯亮起就表示设置成功。



2.3. 开机自启设置

注意：不是所有电脑都可以成功，不同品牌电脑会有差异。

(1) 查看/etc/目录下有没有 rc.local 文件，如果没有则输入指令：`sudo cp rc.local /etc`；如果存在 rc.local 文件，则在 rc.local 的 exit 0 上面加入以下内容后保存。

```
sleep 2
```

```
sudo ip link set can0 type can bitrate 500000
```

```
sudo ip link set can0 up
```



```
1
2
3#!/bin/sh -e
4#
5# rc.local
6#
7# This script is executed at the end of each multiuser runlevel.
8# Make sure that the script will "exit 0" on success or any other
9# value on error.
10#
11# In order to enable or disable this script just change the execution
12# bits.
13#
14# By default this script does nothing.
15
16sleep 2
17
18sudo ip link set can0 type can bitrate 500000
19sudo ip link set can0 up
20
21exit 0
```

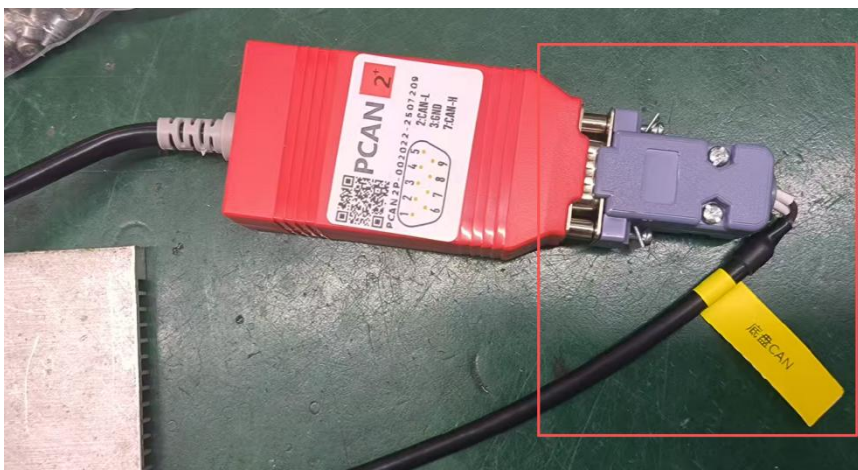
(2) 确认开机或者重启之前，CAN 卡已经接到工控机的 USB 口。

(3) 看到 can 卡绿灯亮起就表示设置成功。



2.4. 通讯测试

(1) 将底盘的 CAN 线与 CAN 卡连接好。



(2) 正确连接好后，可以看到 CAN 卡的绿灯闪烁，如果绿灯不闪烁，检查底盘是否上电，CAN 线是否正确连接，注意（直接将底盘的 CAN 线接到 CAN 卡，确保通讯正常后才能接延长线）。

(3) 安装好 can-utils（输入指令：`sudo apt-get install can-utils`）后，打开终端，输入指令：

`candump can0` 就可以看到 CAN 卡接收到底盘的数据了。

```
yhs@yhs-ros:~$ candump can0
can0 18C4D8EF [8] 00 00 B5 50 05 00 00 E0
can0 18C4D1EF [8] 01 00 00 00 00 00 10 11
can0 18C4D7EF [8] 00 00 20 D7 00 00 10 E7
can0 18C4D8EF [8] 00 00 B5 50 05 00 10 F0
can0 18C4D1EF [8] 01 00 00 00 00 00 20 21
can0 18C4D7EF [8] 00 00 20 D7 00 00 20 D7
can0 18C4D8EF [8] 00 00 B5 50 05 00 20 C0
can0 18C4DAEF [8] 00 10 00 00 00 00 D0 C0
can0 18C4D1EF [8] 01 00 00 00 00 00 30 31
can0 18C4D7EF [8] 00 00 20 D7 00 00 30 C7
can0 18C4D8EF [8] 00 00 B5 50 05 00 30 D0
can0 18C4D1EF [8] 01 00 00 00 00 00 40 41
can0 18C4D7EF [8] 00 00 20 D7 00 00 40 B7
can0 18C4D8EF [8] 00 00 B5 50 05 00 40 A0
can0 18C4D1EF [8] 01 00 00 00 00 00 50 51
can0 18C4D7EF [8] 00 00 20 D7 00 00 50 A7
can0 18C4D8EF [8] 00 00 B5 50 05 00 50 B0
can0 18C4D1EF [8] 01 00 00 00 00 00 60 61
can0 18C4D7EF [8] 00 00 20 D7 00 00 60 97
can0 18C4D8EF [8] 00 00 B5 50 05 00 60 80
can0 18C4E1EF [8] 51 0A A6 FF 06 04 70 70
can0 18C4D1EF [8] 01 00 00 00 00 00 70 71
can0 18C4D7EF [8] 00 00 20 D7 00 00 70 87
```

3. ROS 驱动包使用

使用前提：已经设置好 CAN 卡，并且 CAN 卡与底盘通讯正常。

3.1. 编译

(1) 打开终端，进入 TK-mid-ros2-V1.1.1 文件目录。

(2) 输入命令 `colcon build`，等待编译完成。

```
Starting >>> yhs_can_interfaces
Finished <<< yhs_can_interfaces [10.3s]
Starting >>> yhs_can_control
Finished <<< yhs_can_control [16.4s]
Summary: 2 packages finished [27.0s]
```

3.2. 运行

(1) 打开终端，进入 TK-mid-ros2-V1.1.1 文件目录，分别输入以下命令后敲回车

```
source install/setup.bash
```

```
ros2 launch yhs_can_control yhs_can_control.launch.py
```

(2) 终端输出“yhs_can_control_node initialized successfully”则表示打开成功。

```
[yhs_can_control_node-1] process started with pid [5509]
[yhs_can_control_node-1] [INFO] [1679888512.597938093] [yhs_can_control_node]: yhs_can_control_node initialized successfully
```

3.3. 测试

(1) 打开终端，进入 FR-max-ros2-V1.1.1 文件目录，分别输入以下命令后敲回车。

```
source install/setup.bash
```

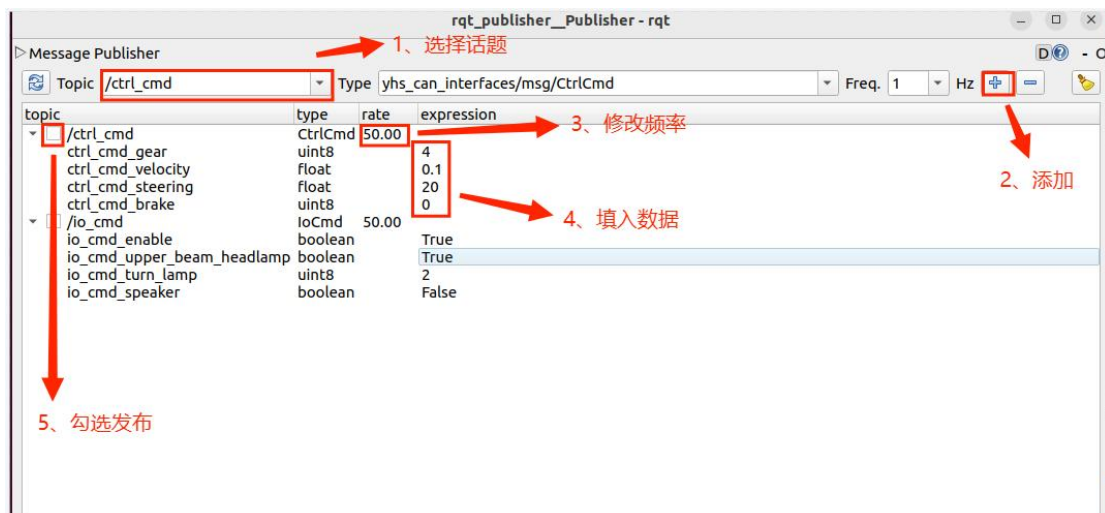
```
ros2 run rqt_publisher rqt_publisher
```

```
$ source install/setup.bash
$
$ ros2 run rqt_publisher rqt_publisher
```

下发指令成功后，can 灯会红绿交替闪烁。

(2) 使用 rqt_publisher 工具时，遥控器要切换到指令控制模式，建议把底盘架起来测试。

注意：打开新终端，需要打印底盘数据时，也要设置一下环境变量。



4. 话题说明

4.1.发布话题

(1) 运动控制，发布频率要在 50hz 以上。

消息类型	yhs_can_interfaces/msg/CtrlCmd	
话题名	ctrl_cmd	
变量	类型	说明
ctrl_cmd_gear	uint8	目标档位 00 : disable 01 : 驻车档 02 : 空挡 03 : 运动学控制档
ctrl_cmd_velocity	float32	目标车体速度 0.001m/s/bit ; 单位 : m/s
ctrl_cmd_steering	float32	目标车体转向角 0.01°/bit ; 单位 : °

(3) IO 控制，发布频率要在 30hz 以上。

消息类型	yhs_can_interfaces/msg/loCmd	
话题名	io_cmd	
变量	类型	说明
io_cmd_lamp_ctrl	bool	灯控制权模式 0 = 根据状态自动控制 1 = 自由控制
io_cmd_unlock	bool	安全停车解锁开关 0 = 无效 1 = 解锁使能
io_cmd_lower_beam_headlamp	bool	近光灯开关 0 = 关闭 1 = 打开

io_cmd_upper_beam_headlamp	bool	远光灯开关 0 = 关闭 1 = 打开
io_cmd_turn_lamp	uint8	转向灯开关 0 = 全关 1 = 开启左转向灯 2 = 开启右转向灯
io_cmd_braking_lamp	bool	制动灯开关 0 = 关闭 1 = 打开
io_cmd_clearance_lamp	bool	示廓灯开关 0 = 关闭 1 = 打开
io_cmd_fog_lamp	bool	雾灯开关 0 = 关闭 1 = 打开
io_cmd_speaker	bool	扬声器开关 0 = 关闭 1 = 打开

4.2. 订阅话题

(1) 底盘反馈。

消息类型	yhs_can_interfaces/msg/ChassisInfoFb	
话题名	chassis_info_fb	
变量	类型	说明
ctrl_fb.ctrl_fb_target_gear	uint8	当前档位反馈 00：disable 01：驻车档 02：空挡 03：运动学控制档
ctrl_fb.ctrl_fb_linear	float32	当前车体线速度反馈 单位：m/s

ctrl_fb.ctrl_fb_angular	float32	当前车体角速度反馈 单位：°/s
io_fb.io_fb_lamp_ctrl	bool	灯控制权状态反馈 0 = 根据状态自动控制 1 = 自由控制
io_fb.io_fb_unlock	bool	安全停车解锁状态反馈 0 = 无效 1 = 解锁成功
io_fb.io_fb_lower_beam_headlamp	bool	近光灯开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_upper_beam_headlamp	bool	远光灯开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_turn_lamp	uint8	转向灯开关状态反馈 0 = 全关 1 = 开启左转向灯 2 = 开启右转向灯
io_fb.io_fb_braking_lamp	bool	制动灯开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_clearance_lamp	bool	示廓灯开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_fog_lamp	bool	雾灯开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_speaker	bool	扬声器开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_fl_impact_sensor	bool	前左防撞条开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_fm_impact_sensor	bool	前中防撞条开关状态反馈

		0 = 关闭 1 = 打开
io_fb.io_fb_fr_impact_sensor	bool	前右防撞条开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_rl_impact_sensor	bool	后左防撞条开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_rm_impact_sensor	bool	后中防撞条开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_rr_impact_sensor	bool	后右防撞条开关状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_fl_drop_sensor	bool	前左跌落传感器状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_fm_drop_sensor	bool	前中跌落传感器状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_fr_drop_sensor	bool	前右跌落传感器状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_rl_drop_sensor	bool	后左跌落传感器状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_rm_drop_sensor	bool	后中跌落传感器状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_rr_drop_sensor	bool	后右跌落传感器状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_estop	bool	急停开关状态反馈 0 = 关闭

		1 = 打开
io_fb.io_fb_joypad_ctrl	bool	遥控器控制状态反馈 0 = 关闭 1 = 打开
io_fb.io_fb_charge_state	bool	充电桩状态反馈 0 = 关闭 1 = 打开
l_wheel_fb.l_wheel_fb_velocity	float32	当前左轮速度反馈 单位：m/s
l_wheel_fb.l_wheel_fb_pulse	int32	当前左轮脉冲数反馈 轮子单圈 N 个脉冲,N= 编码器线束*减速比
r_wheel_fb.r_wheel_fb_velocity	float32	当前右轮速度反馈 单位：m/s
r_wheel_fb.r_wheel_fb_pulse	int32	当前右轮脉冲数反馈 轮子单圈 N 个脉冲,N= 编码器线束*减速比
bms_flag_fb.bms_flag_fb_soc	uint8	当前剩余电量百分比
bms_flag_fb.bms_flag_fb_single_ov	bool	单体过压保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_single_uv	bool	单体欠压保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_ov	bool	整组过压保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_uv	bool	整组欠压保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_charge_ot	bool	充电过温保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_charge_ut	bool	充电低温保护

		0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_discharge_ot	bool	放电过温保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_discharge_ut	bool	放电低温保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_charge_oc	bool	充电过流保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_discharge_oc	bool	放电过流保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_short	bool	短路保护 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_ic_error	bool	前端检测 IC 错误 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_lock_mos	bool	软件锁定 MOS 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_charge_flag	bool	充电标志位 0 = 关闭 1 = 打开
bms_flag_fb.bms_flag_fb_hight_temperature	float32	当前电池最高温度 °C
bms_flag_fb.bms_flag_fb_low_temperature	float32	当前电池最低温度 °C
bms_fb.bms_fb_voltage	float32	当前电池电压 单位：V
bms_fb.bms_fb_current	float32	当前电池电流 单位：A

bms_fb.bms_fb_remaining_capacity	float32	当前电池剩余容量 单位 : Ah
ultrasonic.ultrasonic_fb_01	uint16	超声波雷达探头 1 距离反馈
ultrasonic.ultrasonic_fb_02	uint16	超声波雷达探头 2 距离反馈
ultrasonic.ultrasonic_fb_03	uint16	超声波雷达探头 3 距离反馈
ultrasonic.ultrasonic_fb_04	uint16	超声波雷达探头 4 距离反馈
ultrasonic.ultrasonic_fb_05	uint16	超声波雷达探头 5 距离反馈
ultrasonic.ultrasonic_fb_06	uint16	超声波雷达探头 6 距离反馈
ultrasonic.ultrasonic_fb_07	uint16	超声波雷达探头 7 距离反馈
ultrasonic.ultrasonic_fb_08	uint16	超声波雷达探头 8 距离反馈

5. 注意事项 Attention

在 yhs_can_control.cpp 中，对于功能预留项已进行了注释（如下图所示），若用户所购买的产品含有这些功能，请自行去掉注释。

```
46 // 近光灯开关（预留）
47 // sendData_u_io_[1] = 0xff;
48 // if (msg.io_cmd_lower_beam_headlamp)
49 //   sendData_u_io_[1] &= 0xff;
50 // else
51 //   sendData_u_io_[1] &= 0xfe;
52
53 // 远光灯开关（预留）
54 // if (msg.io_cmd_upper_beam_headlamp)
55 //   sendData_u_io_[1] &= 0xff;
56 // else
57 //   sendData_u_io_[1] &= 0xfd;
58
59 if (msg.io_cmd_turn_lamp == 0)
60   sendData_u_io_[1] &= 0xf3;
61 if (msg.io_cmd_turn_lamp == 1)
62   sendData_u_io_[1] &= 0xf7;
63 if (msg.io_cmd_turn_lamp == 2)
64   sendData_u_io_[1] &= 0xfb;
65
66 // 制动灯开关（预留）
67 // if (msg.io_cmd_braking_lamp)
68 //   sendData_u_io_[1] &= 0xff;
69 // else
70 //   sendData_u_io_[1] &= 0xef;
71
72 if (msg.io_cmd_clearance_lamp)
73   sendData_u_io_[1] &= 0xff;
74 else
75   sendData_u_io_[1] &= 0xdf;
76
77 // 雾灯开关（预留）
78 // if (msg.io_cmd_fog_lamp)
79 //   sendData_u_io_[1] &= 0xff;
80 // else
81 //   sendData_u_io_[1] &= 0xbf;
```

电话：0755-28468956

传真：0755-28468956

网 址 www.yuhesen.com 邮件：sales01@yuhesen.com

地址：深圳市龙岗区宝龙六路 1 号创维群欣科技园 1 栋 5 楼

The logo for YUHESEN, featuring the company name in a bold, blue, sans-serif font.

深圳煜禾森科技有限公司